

Unidad 1. Arquitecturas Web

1. Introducción

Las arquitecturas web constituyen el fundamento sobre el cual se construyen las aplicaciones y servicios que utilizamos diariamente en Internet. Comprender estos modelos arquitectónicos es esencial para cualquier profesional del desarrollo y despliegue de aplicaciones web, ya que determinan aspectos cruciales como el rendimiento, la escalabilidad, la seguridad y el mantenimiento de los sistemas.

Una arquitectura web define la estructura y organización de los componentes que conforman una aplicación, estableciendo cómo se comunican entre sí, cómo se distribuyen las responsabilidades y cómo se gestionan los recursos. La elección de una arquitectura adecuada puede marcar la diferencia entre un sistema robusto y escalable, y uno que presenta problemas constantes de rendimiento y mantenibilidad.

2. Arquitectura Cliente-Servidor

¿Qué es y por qué es importante?

La arquitectura cliente-servidor es como el sistema nervioso de Internet. Es el modelo que hace posible que podamos navegar por páginas web, ver vídeos en streaming, revisar nuestro correo electrónico o comprar online. Podemos pensar en ella como una conversación organizada entre dos partes: una que pide cosas (el cliente) y otra que las proporciona (el servidor).

Este modelo es tan exitoso que prácticamente todas las aplicaciones web que usamos a diario funcionan bajo este principio: desde redes sociales hasta plataformas de trabajo colaborativo.

Los dos protagonistas: Cliente y Servidor

El Cliente: Tu ventana al mundo digital

El cliente es la parte de la aplicación con la que interactúas directamente. En la mayoría de los casos, es tu navegador web (Chrome, Firefox, Safari, Edge, etc.). Piensa en el cliente como un mensajero personal: su trabajo es tomar tus peticiones, enviarlas al lugar correcto y mostrarte las respuestas de forma comprensible.

¿Qué hace exactamente el cliente?

- Presenta la interfaz visual que ves en tu pantalla
- Captura tus acciones (clics, escritura, desplazamientos)
- Envía solicitudes al servidor cuando necesitas información
- Recibe y muestra los resultados de manera atractiva y funcional
- Se ejecuta directamente en tu dispositivo (ordenador, móvil, tablet)

El Servidor: El cerebro detrás de la operación

El servidor es un ordenador especializado que funciona las 24 horas del día esperando peticiones. No tiene pantalla ni teclado visible; su única misión es procesar solicitudes y

enviar respuestas. Es como la cocina de un restaurante: tú no la ves, pero ahí es donde se prepara todo lo que necesitas.

¿Qué hace el servidor?

- Espera y recibe solicitudes de múltiples clientes simultáneamente
- Ejecuta la lógica de negocio (cálculos, validaciones, reglas)
- Consulta y actualiza bases de datos
- Procesa información y genera respuestas personalizadas
- Gestiona la seguridad y los permisos de acceso
- Almacena los datos de forma permanente y segura

¿Cómo se comunican?

La comunicación entre cliente y servidor sigue un protocolo establecido llamado HTTP (Protocolo de Transferencia de HiperTexto). Es como un idioma común que ambos entienden perfectamente.

El ciclo de una petición: un ejemplo práctico

Imaginemos que entras en una red social y quieres ver tu feed de noticias:

1. **Tú (usuario) actúas:** Abres la aplicación o recargas la página
2. **El cliente envía la petición:** Tu navegador envía un mensaje al servidor diciendo "Dame las últimas publicaciones del usuario123"
3. **El servidor procesa:**
 - Verifica tu identidad (¿estás conectado?)
 - Consulta la base de datos buscando publicaciones recientes
 - Aplica filtros según tus preferencias
 - Ordena las publicaciones por relevancia o fecha
4. **El servidor responde:** Envía los datos de las publicaciones al cliente
5. **El cliente muestra el resultado:** Tu navegador organiza y presenta esas publicaciones de forma visual y atractiva en tu pantalla

Todo este proceso suele durar apenas fracciones de segundo.

2.1.Ventajas de este modelo

Separación clara de responsabilidades

Cada componente tiene su función específica. El cliente se preocupa de la presentación y la interacción con el usuario, mientras el servidor maneja la lógica compleja y los datos. Es como tener especialistas para cada tarea.

Centralización de datos y lógica

Toda la información importante y las reglas de negocio están en el servidor. Esto significa que:

- Los datos están seguros y respaldados en un lugar controlado
- Las actualizaciones se hacen una sola vez en el servidor, no en miles de dispositivos
- Todos los usuarios trabajan con la misma versión de la aplicación

Escalabilidad

Cuando más personas empiezan a usar tu aplicación, puedes añadir más servidores para distribuir la carga de trabajo, sin que los usuarios noten ningún cambio.

Mantenimiento independiente

Puedes actualizar el diseño del cliente sin tocar el servidor, o mejorar la lógica del servidor sin cambiar la interfaz. Son piezas independientes que funcionan juntas.

Acceso multiplataforma

El mismo servidor puede atender a clientes muy diferentes: navegadores web, aplicaciones móviles, smartwatches, asistentes de voz, etc. Todos hablan el mismo idioma (HTTP) con el servidor.

Un ejemplo cotidiano: Comprar un libro online

Para entenderlo mejor, veamos qué sucede cuando compras un libro en una tienda online:

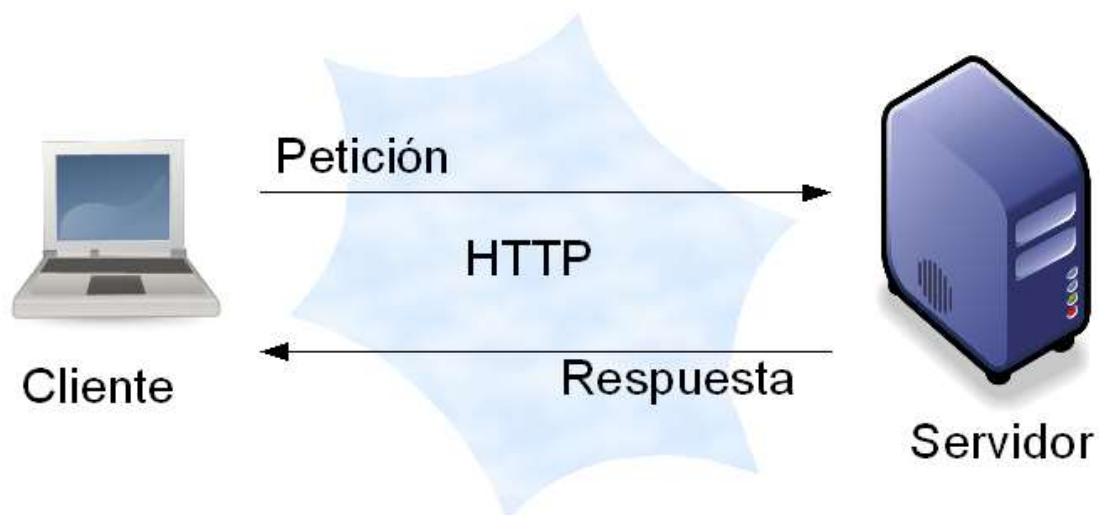
1. **Buscas el libro** → El cliente envía tu búsqueda al servidor
2. **Ves resultados** → El servidor consulta su catálogo y devuelve coincidencias
3. **Añades al carrito** → El cliente informa al servidor, que guarda temporalmente tu selección
4. **Procedes al pago** → El cliente muestra un formulario; tú introduces tus datos
5. **Confirmas la compra** → El servidor verifica el pago, actualiza el inventario, genera un pedido y te envía una confirmación
6. **Recibes confirmación** → El cliente muestra el mensaje de éxito y los detalles de tu pedido

En cada paso, cliente y servidor colaboran: uno presenta y recoge información, el otro procesa y toma decisiones.

2.2. Consideraciones modernas

Aunque el modelo básico sigue siendo el mismo, las aplicaciones modernas han evolucionado para ser más sofisticadas:

- **Clientes más inteligentes:** Los navegadores actuales pueden realizar operaciones complejas localmente, reduciendo la necesidad de consultar constantemente al servidor
- **Comunicación en tiempo real:** Tecnologías como WebSockets permiten que servidor y cliente mantengan una conexión abierta para actualizaciones instantáneas (chats, notificaciones)
- **Múltiples servidores:** Las aplicaciones grandes no usan un solo servidor, sino redes de servidores especializados trabajando en conjunto
- **Cachés intermedias:** Se almacenan copias temporales de datos frecuentes para acelerar las respuestas



3. Arquitectura de Tres Capas

¿Por qué tres capas y no dos?

Imagina que estás construyendo una casa. Podrías simplemente levantar paredes y poner un tejado (dos capas: estructura y cubierta). Pero los arquitectos saben que es mejor añadir una capa intermedia: cimientos, estructura habitable y tejado. Cada una tiene su función específica y pueden mejorarse o repararse independientemente.

La arquitectura de tres capas aplica este mismo principio al desarrollo de software. Es una evolución natural del modelo cliente-servidor básico que vimos anteriormente, pero con una organización más sofisticada y profesional. En lugar de tener simplemente "cliente" y "servidor", dividimos el trabajo en tres niveles especializados, cada uno con responsabilidades muy específicas.

Las tres capas: una orquesta bien coordinada

Piensa en las tres capas como los departamentos de una empresa bien organizada:

Capa de Presentación: El escaparate

¿Qué es? Es todo lo que el usuario ve y toca. La cara visible de tu aplicación. Es como el mostrador de atención al cliente en una tienda: el lugar donde interactúas directamente con el servicio.

¿Qué incluye?

- La interfaz gráfica: botones, formularios, menús, imágenes
- Las páginas web que ves en tu navegador
- Las pantallas de aplicaciones móviles
- Incluso aplicaciones de escritorio o interfaces de voz

¿Cuál es su trabajo?

- Mostrar la información de forma atractiva y comprensible
- Capturar lo que el usuario quiere hacer (clics, toques, comandos de voz)
- Validar que los datos introducidos tengan sentido básico (por ejemplo, que un email tenga formato de email)
- Enviar las solicitudes del usuario a la siguiente capa
- Recibir respuestas y presentarlas de forma visual

Ejemplo práctico: Cuando reservas un vuelo online, la capa de presentación te muestra el calendario para elegir fechas, el formulario para introducir tus datos y el botón para confirmar la reserva. Es todo lo visual y interactivo.

Capa de Lógica de Negocio: El cerebro de la operación

¿Qué es? Es el corazón inteligente de la aplicación. Aquí es donde viven todas las reglas, decisiones y procesos que hacen que tu aplicación funcione según las necesidades del

negocio. Es como el gerente de la tienda que toma decisiones según las políticas de la empresa.

¿Qué incluye?

- Las reglas del negocio (¿quién puede hacer qué?)
- Los cálculos y algoritmos (precios, descuentos, recomendaciones)
- La validación profunda de datos (¿este usuario tiene crédito suficiente?)
- La coordinación de operaciones complejas
- La aplicación de permisos y seguridad

¿Cuál es su trabajo?

- Recibir solicitudes de la capa de presentación
- Aplicar todas las reglas de negocio relevantes
- Realizar cálculos y procesamiento de información
- Decidir qué datos necesita consultar o guardar
- Coordinar con la capa de datos para obtener o almacenar información
- Devolver resultados procesados a la capa de presentación

Ejemplo práctico: Siguiendo con la reserva de vuelo, esta capa verifica que hay plazas disponibles, calcula el precio según la temporada y tu perfil de cliente, aplica descuentos si tienes un código promocional, comprueba que tu tarjeta tiene fondos suficientes y decide si la reserva puede confirmarse o no.

Capa de Acceso a Datos: El archivo y almacén

¿Qué es? Es el guardián de toda la información permanente. Esta capa se encarga exclusivamente de guardar y recuperar datos. Es como el almacén y el archivo de la empresa: todo está ordenado, catalogado y listo para ser consultado o actualizado.

¿Qué incluye?

- Las conexiones a bases de datos
- Las consultas para buscar información
- Las operaciones de guardado y actualización
- Los mecanismos de respaldo y recuperación
- La optimización del almacenamiento

¿Cuál es su trabajo?

- Recibir peticiones de la capa de lógica de negocio para obtener o guardar datos
- Traducir esas peticiones al lenguaje de la base de datos
- Ejecutar las consultas de forma eficiente

- Asegurar que los datos se guarden correctamente
- Mantener la integridad y consistencia de la información
- Devolver los datos solicitados en un formato utilizable

Ejemplo práctico: Esta capa busca en la base de datos los vuelos disponibles, recupera tu historial de reservas, guarda tu nueva reserva con todos los detalles, actualiza el número de plazas disponibles y registra la transacción de pago. Todo queda guardado de forma segura y permanente.

Cómo trabajan juntas: Un ejemplo completo

Veamos qué ocurre cuando compras un producto en una tienda online, paso a paso, capa por capa:

Paso 1: Buscas un producto

- **Presentación:** Escribes "zapatillas running" en el buscador y pulsas Enter
- **Lógica:** Recibe tu búsqueda, la procesa (corrige faltas, añade sinónimos) y pide a Datos que busque productos relacionados
- **Datos:** Consulta la base de datos y recupera todos los productos que coinciden
- **Lógica:** Ordena los resultados según relevancia, stock y valoraciones
- **Presentación:** Te muestra una lista atractiva con fotos, precios y valoraciones

Paso 2: Añades al carrito

- **Presentación:** Haces clic en "Añadir al carrito"
- **Lógica:** Verifica que el producto tiene stock disponible y que el precio es el actual
- **Datos:** Comprueba la disponibilidad en la base de datos
- **Lógica:** Si todo está bien, guarda el producto en tu carrito temporal
- **Presentación:** Te muestra una confirmación y actualiza el icono del carrito

Paso 3: Finalizas la compra

- **Presentación:** Introduces tu dirección y forma de pago, y confirmas
- **Lógica:** Valida que los datos sean correctos, calcula el total con impuestos y gastos de envío, verifica que hay stock, procesa el pago y genera el pedido
- **Datos:** Guarda el pedido completo, actualiza el inventario, registra el pago y almacena la dirección de envío
- **Lógica:** Confirma que todo se guardó correctamente y envía un email de confirmación
- **Presentación:** Te muestra la página de "Pedido confirmado" con todos los detalles

¿Por qué tres capas?

Podrías preguntarte: "¿No sería más simple tenerlo todo junto?". La respuesta es sí, sería más simple... pero solo al principio. A largo plazo, esta separación aporta beneficios enormes:

1. Mantenimiento más fácil

Cada capa puede evolucionar y mejorarse sin afectar a las demás. Es como tener habitaciones separadas en una casa: puedes redecorar el salón sin tocar la cocina.

Ejemplo real: Quieres cambiar el diseño visual de tu web porque se ha quedado anticuado. Con tres capas, solo modificas la capa de presentación. Toda la lógica de negocio y los datos siguen igual. No hay riesgo de romper funcionalidades críticas.

2. Reutilización de código

La misma lógica de negocio puede servir a múltiples interfaces diferentes.

Ejemplo real: Tienes una web de comercio electrónico y decides lanzar también una app móvil. La lógica de negocio (cálculos de precios, gestión de pedidos, validaciones) es exactamente la misma. Solo necesitas crear una nueva capa de presentación para móvil. Ahorras meses de desarrollo y evitas inconsistencias.

3. Equipos especializados

Diferentes desarrolladores pueden trabajar en diferentes capas simultáneamente sin estorbarse.

Ejemplo real: Tu equipo de diseñadores y desarrolladores frontend trabaja en mejorar la experiencia de usuario en la capa de presentación. Mientras tanto, otros desarrolladores optimizan los algoritmos de recomendación en la lógica de negocio. Y tu equipo de bases de datos mejora el rendimiento de las consultas en la capa de datos. Los tres equipos avanzan en paralelo.

4. Pruebas más efectivas

Cada capa puede probarse de forma aislada, detectando problemas más rápidamente.

Ejemplo real: Puedes probar que tu lógica de descuentos funciona correctamente sin necesidad de crear toda la interfaz visual. Simplemente alimentas la capa de lógica con datos de prueba y verificas que los cálculos son correctos. Esto acelera enormemente el proceso de control de calidad.

5. Cambios de tecnología sin drama

Puedes cambiar la tecnología de una capa sin afectar a las otras.

Ejemplo real: Tu base de datos actual se queda pequeña y decides migrar a una más potente. Como la capa de datos está aislada, puedes cambiarla completamente. La capa de lógica ni se entera: sigue pidiendo y recibiendo datos exactamente igual.

6. Escalabilidad independiente

Cada capa puede crecer a su propio ritmo según las necesidades.

Ejemplo real: Tu aplicación tiene mucho éxito y recibe millones de visitas. Notas que la capa de lógica de negocio es la que más se carga. Puedes añadir más servidores solo para esa capa, sin necesidad de multiplicar todo el sistema. Esto optimiza costos y recursos.

Reglas de oro para que funcione

Para que esta arquitectura sea efectiva, hay que seguir algunas reglas importantes:

Comunicación unidireccional

Las capas se comunican siempre en orden: Presentación → Lógica → Datos. La capa de presentación nunca debe hablar directamente con la de datos, saltándose la lógica. Sería como que un cliente entrara en el almacén sin pasar por el mostrador: caos asegurado.

Cada capa en su rol

La presentación no hace cálculos de negocio, la lógica no decide colores de botones, y los datos no validan reglas de negocio. Cada una se centra en lo suyo.

Interfaces claras

Las capas se comunican mediante "contratos" bien definidos. Cada capa sabe exactamente qué puede pedir a la siguiente y qué va a recibir, sin sorpresas.

Casos de uso ideales

Esta arquitectura es especialmente útil para:

- **Aplicaciones empresariales** con reglas de negocio complejas
- **Proyectos grandes** con múltiples equipos de desarrollo
- **Sistemas que necesitarán evolucionar** durante años
- **Aplicaciones multiplataforma** (web, móvil, escritorio)
- **Proyectos donde la lógica de negocio es el valor principal** del producto

Comparación visual: 2 vs 3 capas

Arquitectura de 2 capas (Cliente-Servidor):

Cliente (presenta + algo de lógica) ↔ Servidor (lógica + datos mezclados)

Arquitectura de 3 capas:

Presentación → Lógica de Negocio → Acceso a Datos

(muestra info) (toma decisiones) (guarda y recupera)

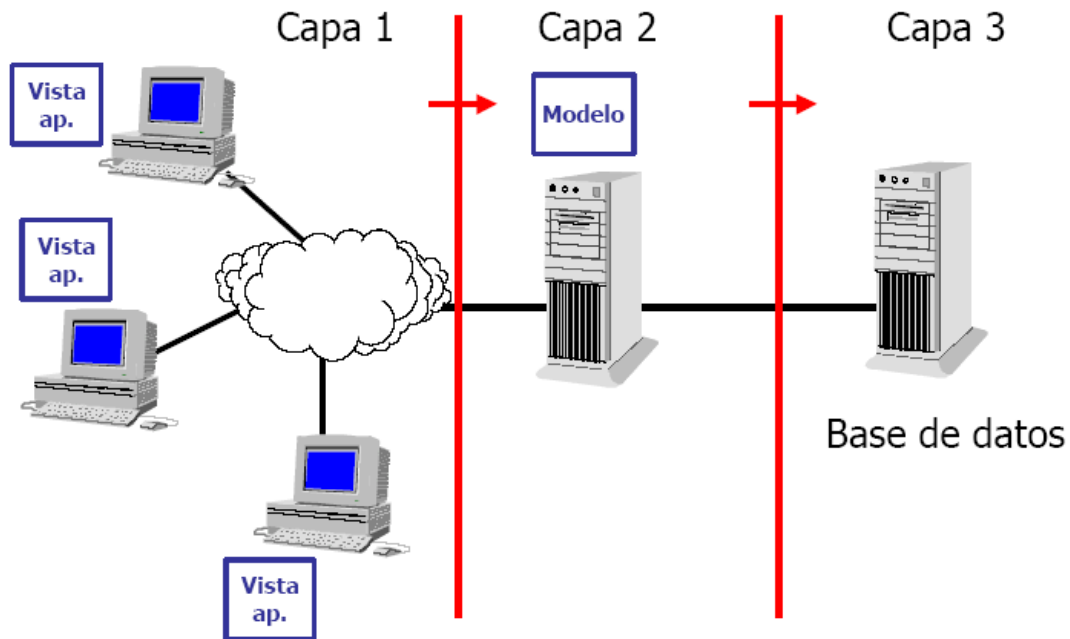
La diferencia es como pasar de una oficina de dos habitaciones a una de tres: más organizada, más profesional y más eficiente a largo plazo.

Conclusión

La arquitectura de tres capas no es solo una forma elegante de organizar código. Es una filosofía de diseño que reconoce que las aplicaciones complejas necesitan estructura, especialización y separación de responsabilidades. Al igual que un edificio bien diseñado

tiene cimientos, estructura y acabados diferenciados, una aplicación profesional beneficia enormemente de tener sus capas de presentación, lógica y datos claramente separadas.

Esta inversión inicial en organización se paga con creces en facilidad de mantenimiento, capacidad de evolución y calidad general del software. Es la diferencia entre construir algo que funciona hoy y construir algo que seguirá funcionando y mejorando durante años.



4. Arquitectura Orientada a Servicios (SOA)

¿Qué es?

Imagina que en vez de tener una aplicación gigante que hace todo (como una navaja suiza), la divides en pedacitos independientes llamados "servicios". Cada pedacito hace una cosa específica y la hace muy bien.

¿Cómo funciona?

Piensa en una empresa grande:

- Cada servicio es como un departamento: Ventas, Contabilidad, Almacén... Cada uno tiene su función clara
- Trabajan de forma independiente: Si Contabilidad cambia su forma de trabajar, Ventas sigue funcionando igual
- Se comunican cuando es necesario: Ventas le dice a Almacén "necesito este producto" y Almacén responde

Características clave:

- Autónomos: Cada servicio puede funcionar por su cuenta

- Bajo acoplamiento: No dependen mucho unos de otros, están poco "enganchados"
- Alta cohesión: Dentro de cada servicio, todo está muy relacionado y organizado
- Reutilizables: Puedes usar el mismo servicio en diferentes partes del sistema

Comunicación entre servicios

Los servicios "hablan" entre sí usando reglas estándar como SOAP (es como hablar todos el mismo idioma).

Ejemplo práctico: Imagina una tienda online:

- Servicio de Usuarios (gestiona cuentas)
- Servicio de Productos (catálogo)
- Servicio de Pagos (procesa compras)
- Servicio de Envíos (gestiona entregas)

Cuando compras, el Servicio de Pagos habla con el de Usuarios para verificar tu cuenta, luego con el de Productos para confirmar el stock, y finalmente con el de Envíos para organizar la entrega.

5. Arquitectura de Microservicios

¿Qué es?

Es como SOA, pero llevado al extremo. Si SOA son departamentos de una empresa, los microservicios son como tener freelancers ultra-especializados. Cada uno hace UNA cosa muy específica y pequeña.

Diferencias con SOA

SOA tradicional:

- Servicios más grandes (por ejemplo, todo el sistema de pedidos)
- Menos servicios pero más complejos
- Suelen compartir bases de datos

Microservicios:

- Servicios diminutos (por ejemplo, solo calcular descuentos)
- Muchos servicios pero muy simples
- Cada uno tiene su propia base de datos

Características principales

1. Independencia total:

- Cada microservicio puede estar programado en un lenguaje diferente

- Uno en Python, otro en Java, otro en JavaScript... ¡no importa!
- Cada equipo elige sus herramientas favoritas

2. Despliegue independiente:

- Puedes actualizar el microservicio de "calcular envíos" sin tocar nada más
- Es como cambiar una rueda del coche sin tener que revisar el motor

3. Equipos pequeños:

- Cada microservicio lo gestiona un equipo reducido (regla de "2 pizzas": el equipo debe poder comer con 2 pizzas)
- Menos burocracia, más agilidad

¿Cómo se comunican?

Usan formas ligeras y rápidas:

- HTTP/REST: Como las APIs de internet que usas todos los días
- Mensajería asíncrona: Como enviar mensajes de WhatsApp en vez de llamar por teléfono

Ventajas (lo bueno)

1. Escalabilidad inteligente: Si tu servicio de pagos recibe mucho tráfico en Black Friday, solo escalas ese servicio, no toda la aplicación. Es como poner más cajas en el supermercado solo los sábados.

2. Resiliencia mejorada: Si el servicio de recomendaciones se cae, puedes seguir comprando. Es como si en un centro comercial se estropea el ascensor pero las escaleras siguen funcionando.

3. Flexibilidad tecnológica: ¿Salió una nueva base de datos mejor? Pruébala en un solo microservicio. Si no funciona, solo afecta a ese pedacito.

4. Despliegues más rápidos: En vez de desplegar toda la aplicación (que puede tardar horas), despliegas un microservicio en minutos.

Desafíos (lo complicado)

1. Complejidad operacional: En vez de vigilar 1 aplicación, ahora vigilas 50 microservicios. Es como pasar de cuidar un perro a cuidar un rebaño de ovejas.

2. Transacciones distribuidas: Si una compra involucra 5 microservicios y el 4º falla, ¿cómo deshaces los 3 primeros? Es complicado.

3. Necesitas muchas herramientas:

- Sistemas para que los servicios se encuentren entre ellos
- Herramientas para ver logs de todos a la vez
- Estrategias para cuando algo falla (circuit breakers, retry logic...)

4. Más recursos: 50 microservicios pequeños pueden gastar más memoria y CPU que 1 aplicación grande.

Ejemplo real

Netflix usa microservicios:

- Un servicio para recomendaciones
- Otro para manejar tu lista
- Otro para streaming de vídeo
- Otro para subtítulos
- Y así tienen cientos...

Si el servicio de recomendaciones falla, sigues viendo tus series sin problema.

6. Arquitectura Serverless (Sin Servidor)

¿Qué es realmente?

"Serverless" es un nombre confuso porque SÍ hay servidores, pero tú no los ves ni te preocupas por ellos. Es como usar Uber en vez de tener tu propio coche: el coche existe, pero no es tu problema.

¿Cómo funciona?

Modelo tradicional (con servidor):

- Alquilas un servidor que está encendido 24/7
- Lo pagas todo el mes aunque solo se use 2 horas al día
- Tú instalas todo, actualizas, arreglas problemas...

Modelo serverless:

- Escribes solo tu código en forma de "funciones"
- Amazon/Google/Microsoft se encargan de ABSOLUTAMENTE TODO lo demás
- Tu función se ejecuta solo cuando alguien la necesita
- El resto del tiempo "no existe" (no pagas nada)

Ejemplo súper simple

Imaginas una función que redimensiona imágenes:

Tradicional:

- Servidor encendido 24/7 esperando imágenes
- Pagas \$50/mes aunque solo suban 10 fotos

Serverless:

- Suben una foto → tu función se ejecuta → redimensiona la imagen → se apaga
- Te cobran \$0.0001 por cada imagen procesada
- Si no suben fotos ese día = \$0

Características importantes

1. Stateless (sin memoria): Cada vez que tu función se ejecuta, empieza de cero. No recuerda nada de la vez anterior. Es como el pez Dory de Buscando a Nemo.

Si necesitas recordar algo, lo guardas en:

- Una base de datos
- Un sistema de caché (Redis)
- Un almacenamiento de archivos (S3)

2. Event-driven (impulsado por eventos): Tu función se ejecuta cuando pasa algo:

- Alguien sube un archivo → procésalo
- Llega una petición HTTP → responde

3. Escalado automático:

- 1 persona usando tu app → 1 función ejecutándose
- 1.000 personas → 1.000 funciones ejecutándose en paralelo
- Tú no haces nada, pasa automáticamente

¿Cuándo usarlo?

Perfecto para:

- APIs con tráfico impredecible: de repente te hacen viral en redes
- Procesamiento de imágenes/vídeos: cuando alguien sube algo
- Tareas programadas: enviar emails cada lunes a las 9 AM
- Chatbots: solo trabajan cuando alguien escribe

NO uses serverless para:

- Aplicaciones con tráfico constante: si está activa 24/7, sale más caro
- WebSockets/conexiones largas: las funciones serverless tienen límite de tiempo
- Procesamiento pesado continuo: render de vídeos de 2 horas
- Latencia crítica: las funciones "frías" tardan unos segundos en arrancar

Proveedores principales

- AWS Lambda (Amazon)
- Google Cloud Functions

- Azure Functions (Microsoft)
- Cloudflare Workers

7. Aplicaciones de Página Única (SPA)

¿Qué es?

Una SPA es una web que funciona como una aplicación de móvil. Entrás una vez, se carga todo, y después todo va rapidísimo sin recargar la página nunca más.

Diferencia con webs tradicionales

Web tradicional (como funcionaba antes):

1. Clickeas en "Productos"
2. El navegador pide la página completa al servidor
3. El servidor genera el HTML entero
4. Te lo envía todo
5. Tu navegador borra todo y carga la nueva página
6. Pantalla blanca, esperas 1-2 segundos
7. ¡Por fin aparece!

SPA (moderna):

1. Primera vez que entras: descarga TODO el código de la aplicación
2. Clickeas en "Productos"
3. JavaScript cambia solo lo necesario en la pantalla
4. Solo pide los datos al servidor (formato JSON, muy ligero)
5. Actualiza la pantalla instantáneamente
6. Sin esperas, sin pantallas en blanco

¿Cómo funciona por dentro?

La carga inicial: Cuando entras por primera vez, descargas:

- El HTML base (muy básico, casi vacío)
- Todo el JavaScript de la aplicación
- Todo el CSS (estilos)
- Las imágenes básicas

Es como instalarte la app, pero en el navegador.

Después: Tu navegador tiene toda la "app" y solo pide datos:

- "Dame la lista de productos" → servidor responde con JSON
- "Dame los detalles del usuario" → servidor responde con JSON
- JavaScript toma esos datos y los muestra

El trabajo del navegador vs el servidor

Antes (web tradicional):

- Servidor: hace TODO (genera HTML, gestiona rutas, renderiza vistas...)
- Navegador: solo muestra lo que le dan

Ahora (SPA):

- Servidor: solo envía datos (API REST)
- Navegador: hace TODO (rutas, vistas, interacciones, animaciones...)

Es como pasar de un restaurante con servicio de mesa a un buffet libre donde tú te sirves.

Ventajas

1. Experiencia de usuario superior:

- Sin recargas: como usar Instagram, todo fluye
- Transiciones suaves: animaciones entre pantallas
- Respuesta instantánea: clickeas y pasa al momento
- Se siente profesional: como una app de verdad

2. Menos carga en el servidor: El servidor solo envía datos, no genera HTML complejo. Ahorra recursos.

3. Desarrollo móvil más fácil: El mismo código puede usarse para hacer apps móviles (con React Native, por ejemplo).

Desafíos y soluciones

1. SEO (que Google te encuentre): Problema: Google tradicionalmente lee HTML, pero en SPAs el HTML inicial está vacío.

Soluciones:

- Server-Side Rendering (SSR): generar el HTML en el servidor la primera vez
- Static Site Generation: pre-generar todas las páginas
- Google es más listo ahora: ya ejecuta JavaScript y puede indexar SPAs

2. Bundle grande (archivo JavaScript pesado): Problema: si tu app es grande, el JavaScript inicial pesa mucho.

Soluciones:

- Code splitting: dividir el código en pedazos y cargar solo lo necesario
- Lazy loading: cargar secciones solo cuando el usuario las necesita
- Optimización: eliminar código no usado, comprimir...

3. Gestión del estado complejo: Problema: mantener sincronizada toda la información de la app es difícil.

Soluciones:

- Usar librerías especializadas: Redux, MobX, Zustand...
- Hooks de React para estados simples

Ejemplos reales de SPAs

- Gmail: toda la interfaz es una SPA
- Twitter/X: navegas sin recargar nunca
- Spotify Web: la música sigue sonando mientras navegas

¿Cuándo usar SPA?

Ideal para:

- Aplicaciones web interactivas (dashboards, herramientas)
- Apps tipo "webapp" (Gmail)
- Plataformas con mucha interacción del usuario
- Cuando quieres la mejor experiencia de usuario posible

Tal vez no para:

- Blogs simples (WordPress es más fácil)
- Webs con mucho contenido estático
- E-commerce simple donde el SEO es crítico
- Proyectos pequeños donde no compensa la complejidad

8. Patrones de Despliegue

Ya hemos visto que la **arquitectura web** se refiere a cómo está estructurado y organizado un sistema o aplicación web internamente —es decir, cómo se dividen y comunican sus componentes (por ejemplo, cliente-servidor, modelo MVC, microservicios, etc.)—, mientras que el **despliegue** se enfoca en **cómo y dónde** se ejecuta esa aplicación en entornos reales (por ejemplo, en un solo servidor en un **modelo monolítico**, en varios servicios distribuidos con **microservicios**, o en **contenedores** mediante Docker o Kubernetes). En resumen, la arquitectura define el **diseño lógico del software**, y el despliegue define su **implementación física y operativa**.

Despliegue Monolítico

En el despliegue monolítico, toda la aplicación se empaqueta y despliega como una única unidad. Esto simplifica el despliegue inicial y la gestión, pero puede complicar las actualizaciones y el escalado, ya que cualquier cambio requiere redespregar toda la aplicación.

Despliegue Distribuido

El despliegue distribuido divide la aplicación en múltiples componentes que pueden desplegarse independientemente en diferentes servidores o contenedores. Esto permite escalado selectivo y actualizaciones independientes, pero aumenta la complejidad operacional.

Contenedores y Orquestación

Los contenedores, mediante tecnologías como Docker, proporcionan un método estandarizado para empaquetar aplicaciones con todas sus dependencias. Los sistemas de orquestación como Kubernetes gestionan el despliegue, escalado y operación de contenedores en clústeres, facilitando la gestión de aplicaciones distribuidas complejas.

9. Consideraciones de Seguridad

La seguridad no es algo que añades al final, como un candado en la puerta. Debe estar presente en cada capa, en cada decisión de diseño, desde el primer día. Es como construir una casa: no puedes pensar en cerraduras y alarmas solo cuando ya está terminada; la seguridad debe estar integrada en los cimientos, las paredes y el tejado.

En una arquitectura web, la seguridad debe ser transversal: atraviesa todas las capas y todos los componentes. Ninguna capa puede confiar ciegamente en las demás. Es el principio de "confía, pero verifica... mejor aún, no confíes tanto".

Autenticación y Autorización: ¿Quién eres y qué puedes hacer?

Autenticación: Verificando identidades

La autenticación es el proceso de confirmar que alguien es quien dice ser. Es como mostrar tu DNI en un control de seguridad.

Métodos comunes:

- Contraseñas robustas: Combinación de caracteres, números y símbolos, almacenadas siempre cifradas (nunca en texto plano)
- Autenticación de dos factores (2FA): Además de la contraseña, un código temporal en tu móvil o email
- Biometría: Huella dactilar, reconocimiento facial
- Tokens de sesión: Una "credencial temporal" que el servidor te da después de iniciar sesión

Ejemplo práctico: Cuando accedes a tu banco online, introduces tu usuario y contraseña (algo que sabes), luego te envían un código SMS (algo que tienes). Solo combinando ambos factores puedes entrar.

Autorización: Controlando permisos

Una vez que sabemos quién eres, necesitamos saber qué puedes hacer. Es como tener una tarjeta de acceso en una empresa: puede que te deje entrar al edificio, pero no a todas las habitaciones.

Niveles de autorización típicos:

- Usuario básico: Puede ver y editar solo su propio contenido
- Moderador: Puede ver y moderar contenido de otros usuarios
- Administrador: Tiene acceso a configuración y funciones críticas
- Superadministrador: Acceso total al sistema

Ejemplo práctico: En una plataforma de educación online, un estudiante puede ver cursos y hacer exámenes, un profesor puede crear contenido y calificar, pero solo un administrador puede gestionar usuarios y ver estadísticas globales.

Cifrado de Datos: Secretos bien guardados

En tránsito: HTTPS siempre

Toda comunicación entre cliente y servidor debe ir cifrada mediante HTTPS (TLS/SSL). Es como enviar una carta en un sobre sellado en lugar de una postal que cualquiera puede leer.

Qué protege:

- Nombres de usuario y contraseñas
- Datos personales
- Información de pago
- Cualquier comunicación entre tu navegador y el servidor

Cómo implementarlo:

- Obtener un certificado SSL (gratuitos con Let's Encrypt)

- Configurar el servidor para forzar HTTPS
- Activar HSTS para que el navegador siempre use HTTPS

En reposo: Datos protegidos incluso si roban el disco

Los datos sensibles almacenados en bases de datos también deben estar cifrados. Si alguien roba físicamente el servidor o accede al disco duro, no podrá leer la información.

Qué cifrar:

- Siempre: Contraseñas (con algoritmos como bcrypt o Argon2, nunca cifrado simple)
- Datos sensibles: Números de tarjetas, datos médicos, documentos de identidad
- Según regulación: Datos personales si tu jurisdicción lo requiere (GDPR, etc.)

Ejemplo práctico: Una contraseña como "MiContraseña123" no se guarda tal cual en la base de datos. Se guarda como algo así:

\$2y\$10\$N9qo8uLOickgx2ZMRZoMyeljZAgcfl7p92ldGxad68LJZdL17lhWy

Es imposible convertir esto de vuelta a la contraseña original. Cuando el usuario inicia sesión, se cifra lo que escribió y se compara con esto.

Principios Fundamentales de Seguridad

1. Principio de Menor Privilegio

Cada componente, usuario o servicio debe tener únicamente los permisos mínimos necesarios para hacer su trabajo, nada más.

Ejemplos prácticos:

- El código que muestra productos en tu web no necesita permisos para borrar usuarios
- Un empleado de atención al cliente puede ver pedidos pero no modificar precios

Por qué es importante: Si un atacante compromete un componente con privilegios limitados, el daño que puede hacer también es limitado.

2. Defensa en Profundidad

No confíes en una única línea de defensa. Implementa múltiples capas de seguridad, de modo que si una falla, las demás siguen protegiendo.

Capas de defensa típicas:

1. Firewall: Bloquea tráfico sospechoso antes de que llegue al servidor
2. WAF (Web Application Firewall): Filtra peticiones HTTP maliciosas
3. Validación en cliente: Primera comprobación rápida (pero nunca confiar solo en ella)

4. Validación en servidor: Verificación real de los datos
5. Cifrado: Incluso si alguien roba los datos, no puede leerlos
6. Logs y monitorización: Detecta intentos de intrusión en tiempo real
7. Backups: Si todo falla, puedes recuperar

Analogía: Es como proteger tu casa con cerradura en la puerta, alarma, cámara de seguridad y caja fuerte para los objetos de valor. Si un ladrón supera la cerradura, aún tiene que lidiar con la alarma.

3. No Confiar en el Cliente

Nunca asumas que el cliente (navegador, app móvil) sigue las reglas. Un atacante puede modificar cualquier cosa que ocurra en el lado del cliente.

Errores comunes:

- Validar solo en JavaScript del navegador (puede desactivarse)
- Ocultar botones de administrador con CSS (pueden hacerse visibles)
- Confiar en parámetros de URL o cookies sin verificar
- Asumir que los datos llegan en el formato esperado

Regla de oro: Todo lo que viene del cliente debe tratarse como potencialmente malicioso hasta que se verifique en el servidor.

Mantente Actualizado: La Seguridad es un Proceso

La seguridad no es algo que implementas una vez y olvidas. Nuevas vulnerabilidades se descubren constantemente.

Buenas prácticas:

- Actualiza dependencias regularmente: Librerías y frameworks publican parches de seguridad
- Auditorías de seguridad periódicas: Revisa tu código y configuración
- Pruebas de penetración: Contrata expertos para intentar hackear tu sistema
- Monitorización continua: Detecta comportamientos anómalos
- Plan de respuesta a incidentes: Sabe qué hacer si te hackean

Checklist de Seguridad Básica

Antes de lanzar cualquier aplicación web, verifica:

- HTTPS activado y forzado en todas las páginas
- Autenticación robusta implementada
- Sistema de autorización por roles funcionando
- Todas las entradas de usuario validadas en el servidor

- Contraseñas cifradas con algoritmos modernos
- Datos sensibles cifrados en la base de datos
- Principio de menor privilegio aplicado
- Logs de seguridad activados
- Plan de backups y recuperación probado

En resumen

La elección de una arquitectura web apropiada depende de múltiples factores: los requisitos funcionales y no funcionales del proyecto, las restricciones de recursos, la experiencia del equipo de desarrollo y las expectativas de crecimiento futuro. No existe una arquitectura universalmente superior; cada una ofrece compromisos diferentes entre complejidad, rendimiento, escalabilidad y facilidad de mantenimiento.

El profesional del despliegue de aplicaciones web debe comprender estas diferentes arquitecturas, sus ventajas, limitaciones y casos de uso apropiados, para poder tomar decisiones informadas que resulten en sistemas robustos, eficientes y mantenibles a largo plazo.

Bibliografía y referencias

Caballero, F. J., & Vázquez, R. (2022). *Administración de redes y servicios de red*. Madrid: Paraninfo.

López, M., & García, A. (2021). *Configuración y administración de servicios de red en sistemas operativos de servidor*. RA-MA Editorial.

Ministerio de Educación y Formación Profesional. (2020). *Guía de aprendizaje del módulo: Servicios de red e Internet (ASIR)*. Madrid: Gobierno de España.

Redondo, J. M., & Salas, P. (2019). *Infraestructuras de red y servicios en entornos corporativos*. Editorial Marcombo.